

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



FIUSAC
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Sistema de Gestión de Reservas y Boletos Aéreos con Apache Cassandra

PONDERACIÓN: 36 pts

Tiempo estimado: 20 hrs

Índice

1. MARCO FORMATIVO.....	3
1.1. Valor.....	3
1.2. Competencia(s).....	3
2. Resumen Ejecutivo.....	3
3. Enunciado del Proyecto.....	4
3.1 Descripción del problema a resolver.....	4
3.2 Alcance del proyecto.....	5
3.3 Entregables.....	6
4. Metodología.....	7
5. Recursos y herramientas a utilizar.....	8
6. Desarrollo de Habilidades Blandas.....	8
7. Cronograma.....	9
8. Rúbrica de Calificación.....	9

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Responsabilidad	Los estudiantes son responsables de diseñar e implementar correctamente el modelo de datos distribuido en Apache Cassandra, garantizando la disponibilidad, consistencia y eficiencia del sistema de reservas desarrollado.

1.2. Competencia(s)

Tipo de Competencia	
Competencia General	Construye soluciones integrales trabajando en forma colaborativa y propositiva en equipos interdisciplinarios, en forma presencial o utilizando plataformas virtuales.
Competencia Específica	Implementa técnicas de modelado de datos distribuidos en Apache Cassandra para optimizar la estructura de almacenamiento y consulta de información, garantizando la escalabilidad, alta disponibilidad y eficiencia del sistema ante grandes volúmenes de datos.

2. Resumen Ejecutivo

Este proyecto está diseñado para que los estudiantes apliquen de manera integral sus competencias en el diseño, implementación y consulta de bases de datos distribuidas utilizando Apache Cassandra. El desafío central consiste en construir un sistema de gestión de reservas y boletos aéreos capaz de manejar grandes volúmenes de datos con baja latencia y alta disponibilidad.

A partir del análisis de los requerimientos del negocio, los estudiantes deberán diseñar y documentar un modelo de datos optimizado para Cassandra, aplicando estrategias de denormalización controlada, particionamiento eficiente y claves primarias compuestas. Un requisito fundamental de este proyecto es evitar el uso de JOINS y consultas con ALLOW FILTERING, diseñando tablas especializadas según las consultas más frecuentes del sistema.

Una vez definido el modelo, los estudiantes procederán a su implementación en un clúster de al menos 3 nodos de Apache Cassandra. Esta fase no se limita a la creación de tablas; se espera que los estudiantes configuren correctamente el Replication Factor, el Consistency Level y las estrategias de compactación. Además, deberán insertar un volumen significativo de datos y validar el comportamiento del sistema ante fallos de nodos, midiendo el impacto en disponibilidad y rendimiento.

3. Enunciado del Proyecto

3.1 Descripción del problema a resolver

Los estudiantes diseñarán e implementarán una base de datos distribuida en Apache Cassandra, optimizada para la gestión de reservas y venta de boletos de una aerolínea, incluyendo vuelos, aeronaves, asientos, pasajeros y pagos. El problema principal consiste en modelar los datos de forma eficiente para un entorno distribuido, donde las consultas deben ejecutarse con baja latencia sin depender de JOINS ni ALLOW FILTERING sobre grandes volúmenes de datos.

El sistema deberá garantizar baja latencia en consultas mediante una correcta elección de claves de partición, alta disponibilidad con configuración de clúster multi-nodo, eficiencia en inserción y consulta para gestionar múltiples reservas simultáneas, y escalabilidad horizontal asegurando rendimiento ante alta concurrencia.

Las entidades principales del sistema son:

- **Pasajeros:** personas que pueden reservar y comprar boletos. De cada pasajero se debe guardar Nombre, Email, Número de pasaporte o DPI, Teléfono y Nacionalidad.
- **Aeronaves:** aviones utilizados para operar los vuelos. De cada aeronave se debe guardar Modelo, Aerolínea operadora, Número de matrícula y Capacidad máxima de pasajeros.
- **Vuelos:** trayectos programados entre dos aeropuertos. De cada vuelo se debe guardar Código de vuelo, Aeronave asignada, Aeropuerto de origen, Aeropuerto de destino, Fecha y hora de salida, Fecha y hora de llegada, y Estado del vuelo (programado, retrasado, cancelado, completado).
- **Asientos:** unidades de capacidad de un vuelo específico. De cada asiento se debe guardar el Vuelo al que pertenece, Número de asiento, Clase (económica, ejecutiva, primera clase) y Estado (disponible, ocupado, bloqueado).
- **Reservas:** registros de la intención de un pasajero de ocupar un asiento en un vuelo. De cada reserva se debe guardar el Pasajero, el Vuelo, el Asiento asignado, Fecha de la reserva y Estado de la reserva (pendiente, confirmada, cancelada).
- **Pagos:** transacciones asociadas a una reserva. De cada pago se debe guardar la Reserva asociada, Monto, Método de pago, Fecha de pago y Estado del pago (pendiente, pagado, reembolsado).

Reglas de negocio del sistema:

1. Un pasajero puede reservar múltiples vuelos en fechas y rutas distintas.
2. Un asiento de un vuelo específico solo puede estar asignado a un pasajero a la vez.
3. Las reservas pueden modificarse o cancelarse hasta 24 horas antes de la hora de salida del vuelo.
4. El número de reservas confirmadas de un vuelo no puede superar la capacidad máxima de la aeronave asignada (control de sobreventa).

5. Cada reserva debe tener asociado un pago con su estado correspondiente (pendiente, pagado o reembolsado).
6. Si un vuelo es cancelado, el cambio de estado debe reflejarse en todas las reservas y pagos asociados a ese vuelo.
7. Se deben permitir consultas agregadas de ocupación e ingresos (no solo listados simples), como el porcentaje de ocupación por vuelo y el ranking de vuelos por ingresos generados.

3.2 Alcance del proyecto

Alcance obligatorio:

- Diseñar un modelo conceptual entidad-relación que represente la lógica del negocio.
- Transformar el modelo conceptual en estructuras optimizadas para Cassandra, aplicando denormalización controlada y creando tablas especializadas según las consultas más frecuentes.
- Configurar y desplegar un clúster de al menos 3 nodos de Apache Cassandra con Replication Factor adecuado.
- Insertar al menos 100,000 reservas en el sistema utilizando Batch Writes para optimizar la carga masiva.
- Implementar las siguientes consultas eficientes en CQL. No basta con un `SELECT` simple por clave de partición: cada una exige una estrategia de modelado propia (contadores, bucketing, orden por clustering key, agregación) para resolverse sin `ALLOW FILTERING`:
 1. **Disponibilidad de asientos por clase:** para un vuelo específico, obtener el conteo de asientos disponibles y ocupados desglosado por clase (económica, ejecutiva, primera). Requiere mantener una tabla de contadores (`counter`) que se actualice en cada reserva/cancelación, en lugar de contar filas en tiempo de consulta.
 2. **Historial cronológico de un pasajero:** dado un pasajero y un rango de fechas, obtener sus vuelos y reservas ordenados cronológicamente, incluyendo el estado de pago de cada una. Requiere clustering keys por fecha (con `ORDER BY` y comparadores de rango `>=`/`<=`) y una tabla denormalizada que combine datos de reserva y pago.
 3. **Manifiesto de vuelo ordenado y enriquecido:** para un vuelo específico, obtener el listado de pasajeros ordenado por número de asiento, incluyendo clase, estado de la reserva y estado del pago en una sola consulta (sin `JOIN`), lo que obliga a denormalizar la información de tres entidades en una tabla especializada.
 4. **Porcentaje de ocupación por ruta y rango de fechas:** para una ruta (origen-destino), calcular qué porcentaje de la capacidad de cada vuelo fue reservado dentro de un rango de fechas. Requiere bucketing de la partición (por ejemplo, ruta + mes) para evitar particiones gigantes y combinar un contador de reservas confirmadas con la capacidad de la aeronave.
 5. **Top N de vuelos por ingresos generados:** obtener los N vuelos con mayor ingreso (suma de pagos confirmados) dentro de un rango de fechas, ordenados de mayor a menor. Requiere una tabla de agregación de ingresos

por vuelo con clustering key en orden DESC sobre el monto (o un patrón equivalente), de forma que el LIMIT N se resuelva sin ordenar ni filtrar en el cliente.

- Simular fallos de nodos y documentar la recuperación del sistema y la disponibilidad de los datos.
- Evaluar el impacto de diferentes Consistency Levels (ONE, QUORUM, ALL) en rendimiento y confiabilidad.
- Configurar TTL (Time-To-Live) para eliminar reservas expiradas automáticamente.

Alcance opcional:

- Implementar Materialized Views para consultas adicionales optimizadas.
- Aplicar Secondary Indexes donde sea estrictamente necesario y justificado.
- Optimizar estrategias de compactación y caché para mejorar latencias de lectura y escritura.

4.3 Requerimientos técnicos

Los estudiantes utilizarán Apache Cassandra como motor de base de datos distribuido para el diseño, creación y manipulación de los datos. Se podrá utilizar Python para la generación y carga masiva de datos mediante scripts automatizados.

El sistema estará basado en una aerolínea que gestiona vuelos, aeronaves, asientos, pasajeros, reservas y pagos. Los requerimientos técnicos específicos son:

Modelado de datos en Cassandra (Query-Driven Modeling):

El diseño del modelo lógico debe estar basado en queries: primero se identifican todas las consultas que el sistema necesita responder y, a partir de ellas, se definen las tablas. Cada tabla debe estar diseñada para responder una query específica de forma eficiente, aplicando Primary Key compuesta con Partition Key bien definida. Los datos deben estar denormalizados para optimizar consultas sin usar JOINS. Se deben crear múltiples tablas especializadas, una por cada patrón de acceso identificado, para evitar ALLOW FILTERING.

Configuración del clúster:

Implementar un clúster de 3 nodos en entorno local o mediante Docker. Configurar el Replication Factor y el Consistency Level adecuados. Seleccionar y justificar la estrategia de replicación entre SimpleStrategy o NetworkTopologyStrategy.

Insertión y consulta de datos:

Insertar al menos 100,000 reservas usando Batch Writes. Implementar las 5 consultas CQL requeridas por el negocio de forma eficiente. Documentar el diseño de cada tabla según la consulta que soporta.

Pruebas de escalabilidad y tolerancia a fallos:

Simular la caída de un nodo y verificar la recuperación automática. Medir el impacto de los distintos Consistency Levels en rendimiento y consistencia. Documentar resultados con métricas y análisis comparativo.

3.3 Entregables

Tipo	Descripción
Prototipo	Clúster de Apache Cassandra funcional (local o Docker) con todos los datos cargados y las consultas operativas.
Documentación Técnica	Manuales de usuario, guía de instalación, modelo ER, modelo lógico basado en queries para Cassandra (indicando qué query responde cada tabla) y descripción de todo el proceso de diseño y configuración.
Código Fuente	El código completo del proyecto, con una estructura clara y comentada, que refleja las buenas prácticas de programación y uso de control de versiones.
Mockups/Diseños	Modelo ER conceptual y esquema lógico de tablas Cassandra basado en queries, mostrando la relación entre cada consulta del negocio y su tabla correspondiente, con justificación de claves de partición y clustering según los patrones de acceso.
Manual de Usuario	Documento que explica cómo usar el sistema desarrollado, incluyendo las consultas CQL implementadas, pasos de instalación del clúster y resolución de problemas comunes.
Repositorio	Deberán crear un repositorio en GitHub con el nombre BD2_2S26_Proyectos_carnet y crear la carpeta "Primer Proyecto" donde harán la entrega de estos entregables.

4. Metodología

Fase 1: Análisis de Requerimientos y Modelado Conceptual

Los estudiantes analizarán los requerimientos del sistema de reservas aéreas, identificando las entidades principales, sus atributos y las relaciones entre ellas. Diseñarán el modelo entidad-relación conceptual y definirán las consultas más frecuentes que el sistema debe soportar, ya que en Cassandra el modelo se diseña en función de las queries.

Fase 2: Diseño del Modelo Lógico Basado en Queries para Cassandra

Transformarán el modelo conceptual en estructuras optimizadas para Apache Cassandra siguiendo el principio de diseño basado en queries (Query-Driven Modeling): las tablas se diseñan a partir de las consultas que el sistema debe responder, no de las entidades del dominio. Identificarán y documentarán cada query requerida por el negocio antes de definir cualquier tabla, y crearán tablas especializadas para soportar cada tipo de consulta de forma eficiente. Aplicarán denormalización controlada, eliminarán relaciones tradicionales y definirán las claves de partición y clustering de cada tabla justificando su elección en función de la query que soporta.

Fase 3: Configuración del Clúster e Implementación Física

Desplegarán el clúster de 3 nodos de Apache Cassandra en entorno local o Docker. Crearán el keyspace con la estrategia de replicación seleccionada y el Replication Factor adecuado. Implementarán todas las tablas definidas en el modelo lógico mediante scripts CQL.

Fase 4: Carga Masiva de Datos

Desarrollarán scripts en Python o CQL para insertar al menos 100,000 registros de reservas en el sistema, utilizando Batch Writes para optimizar el proceso. Verificarán la distribución de datos entre los nodos del clúster y validarán la integridad de la información cargada.

Fase 5: Implementación y Validación de Consultas

Implementarán las 5 consultas CQL requeridas por el negocio, verificando que funcionen de forma eficiente sin recurrir a ALLOW FILTERING. Documentarán el plan de ejecución de cada consulta y medirán su latencia.

Fase 6: Pruebas de Escalabilidad, Tolerancia a Fallos y Documentación

Realizarán pruebas de tolerancia a fallos simulando la caída de nodos y midiendo el impacto en disponibilidad y rendimiento según diferentes Consistency Levels. Documentarán los resultados obtenidos, casos de prueba, métricas comparativas y conclusiones del proyecto.

5. Recursos y herramientas a utilizar

Se le recomienda al estudiante usar las siguientes herramientas y recursos:

Software de Base de Datos:

- Apache Cassandra (versión 4.x o superior)
- DataStax DevCenter o DBeaver para administración visual
- Docker para despliegue del clúster multi-nodo

Herramientas de Desarrollo:

- Python (para generación y carga masiva de datos)
- Librerías: cassandra-driver, faker, pandas
- cqlsh para ejecución de scripts CQL directamente en el clúster

Herramientas de Modelado:

- Draw.io, Lucidchart o Chroma para diagramas ER y esquemas de tablas Cassandra

Plataformas de Colaboración:

- GitHub para control de versiones del código
- UEDI para entrega.

Lecturas Recomendadas:

- Documentación oficial de Apache Cassandra
- Datastax Academy: Data Modeling for Apache Cassandra
- Documentación oficial del driver de Python para Cassandra.

6. Desarrollo de Habilidades Blandas

6.1. En proyectos:

- **Responsabilidad Integral:** Asunción total del ciclo de vida del proyecto, desde el análisis y diseño del modelo distribuido hasta la configuración del clúster, carga de datos, implementación de consultas y documentación técnica.
- **Capacidad de Autoaprendizaje:** Investigación y resolución independiente de problemas técnicos encontrados durante el proceso de modelado para Cassandra, configuración del clúster y optimización de consultas CQL.
- **Comunicación Escrita y Documentación:** Habilidad para transmitir decisiones técnicas complejas de forma clara y estructurada a través del informe del proyecto, justificando las elecciones de diseño del modelo distribuido.
- **Gestión del Tiempo y Disciplina:** Organización autónoma de las fases de desarrollo para cumplir con los hitos establecidos, asegurando la calidad en cada etapa sin supervisión constante.

7. Cronograma

Tipo	Fecha Inicio	Fecha Fin
Asignación de Proyecto	08/09/26	22/09/26
Calificación	25/09/26	26/09/26

8. Rúbrica de Calificación

Antes de la evaluación del proyecto, los estudiantes deben cumplir con los requisitos que se indiquen en esta sección.

Tema	Descripción	Cumple (Sí/No)
Modelo de datos optimizado	El diseño debe incluir tablas denormalizadas y especializadas por consulta, con claves de partición y clustering correctamente definidas y justificadas para las 6 entidades del sistema.	
Configuración del clúster	El clúster debe estar compuesto por al menos 3 nodos funcionales con Replication Factor y Consistency Level configurados y documentados.	
Carga masiva de datos	El sistema debe contener al menos 100,000 reservas insertadas mediante Batch Writes, con distribución verificada entre nodos.	
Consultas CQL eficientes	Las 5 consultas implementadas deben funcionar sin ALLOW FILTERING y cubrir los casos de uso requeridos: disponibilidad de asientos por clase, historial cronológico de un pasajero, manifiesto de vuelo enriquecido, porcentaje de ocupación por ruta y rango de fechas, y Top N de vuelos por ingresos. Las consultas de agregación (ocupación e ingresos) deben resolverse mediante contadores o tablas de agregación, no calculando en el cliente.	
Pruebas de tolerancia a fallos	Se debe documentar la simulación de caída de nodos y el análisis comparativo del impacto de los Consistency Levels ONE, QUORUM y ALL.	

Base de datos funcional	El clúster de Cassandra debe estar operativo (local o Docker) con todos los datos cargados y las consultas ejecutándose correctamente.	
Documentación técnica obligatoria	Se debe incluir documento técnico en PDF que contenga el modelo ER, el esquema lógico de tablas Cassandra basado en queries (indicando explícitamente qué consulta CQL satisface cada tabla), justificación de claves según los patrones de acceso de cada query y descripción del proceso de diseño y configuración del clúster. Manual de usuario obligatorio: Se debe incluir manual de usuario en PDF que explique las consultas CQL implementadas con ejemplos prácticos, parámetros utilizados y guía de instalación del clúster.	

Hoja de Calificación — Proyecto: Sistema de Reservas en Apache Cassandra

	Criterio/Nivel			Nivel de evaluación		
No.	Criterio de evaluación	Punteo maximo		Satisfactorio 100% - 61%	Necesita mejorar 60% - 0%	Punteo Obtenido
1	Habilidades	40				
1.1	Documentación Técnica	15	Descripción	La documentación presenta el proceso de diseño completo del modelo distribuido en Cassandra, incluyendo el modelo ER conceptual, el esquema lógico de tablas con justificación de claves de partición y clustering, la configuración del clúster y las decisiones de diseño.	La documentación está incompleta, no muestra el modelo ER, carece del esquema lógico de tablas Cassandra, o no justifica la elección de claves de partición y estrategias de replicación.	
			Rango del punteo	15 - 9	8 - 0	

1.2	Manual de Usuario	10	Descripción	El manual explica claramente cómo ejecutar cada consulta CQL implementada, incluye ejemplos prácticos con parámetros y resultados esperados, y contiene una guía de instalación del clúster paso a paso.	El manual es confuso, carece de ejemplos prácticos, no documenta adecuadamente las consultas CQL o no incluye la guía de instalación del clúster.	
			Rango del punteo	10 - 6	5 - 0	
1.3	Código Fuente Organizado	15	Descripción	El código CQL y Python está bien estructurado, comentado, utiliza nomenclatura consistente y está organizado en archivos lógicos (DDL, scripts de carga, consultas). El repositorio en GitHub cumple con la estructura solicitada.	El código está desorganizado, sin comentarios, con nomenclatura inconsistente, difícil de entender o el repositorio no tiene la estructura solicitada.	
			Rango del punteo	15 - 9	8 - 0	
	Sub-Total de Puntos	40				

2	Conocimientos	60				
2.1	Modelo de Datos Optimizado para Cassandra	20	Descripción	El diseño aplica correctamente denormalización controlada, crea tablas especializadas por consulta, define claves de partición y clustering adecuadas y evita por completo el uso de JOINS y ALLOW FILTERING.	El diseño no aplica denormalización, utiliza ALLOW FILTERING, las claves de partición están mal definidas o el modelo no está optimizado para las consultas requeridas del negocio.	
			Rango del punteo	20 - 12	11 - 0	
2.2	Configuración e Implementación del Clúster	15	Descripción	El clúster de 3 nodos está correctamente desplegado (local o Docker) con Replication Factor, Consistency Level y estrategia de replicación configurados, documentados y justificados. Todas las tablas están implementadas y operativas.	La implementación presenta errores, el clúster tiene menos de 3 nodos funcionales, faltan configuraciones de Replication Factor o Consistency Level, o el clúster no se ejecuta correctamente.	
			Rango del punteo	15 - 9	8 - 0	
2.3	Consultas CQL Eficientes	15	Descripción	Se implementan correctamente las 5 consultas requeridas: disponibilidad por clase, historial	Las consultas están incompletas, dependen	

				cronológico de pasajero, manifiesto de vuelo enriquecido, porcentaje de ocupación por ruta/fecha y Top N de vuelos por ingresos. Funcionando sin ALLOW FILTERING, con contadores/tablas de agregación bien diseñados, latencias documentadas y plan de ejecución justificado	de ALLOW FILTERING, no cubren todos los casos de uso requeridos o presentan errores de ejecución.	
			Rango del punteo	20 - 12	11 - 0	
2.4	Carga Masiva de Datos	5	Descripción	Se insertan al menos 100,000 reservas utilizando Batch Writes mediante scripts Python. Se verifica la distribución de datos entre los nodos del clúster y se valida la integridad de la información cargada.	La carga de datos presenta errores, no alcanza los 100,000 registros requeridos, no utiliza Batch Writes o no se verifica la distribución entre nodos.	
			Rango del punteo	5 - 3	2 - 0	
2.5	Pruebas de Tolerancia a Fallos	5	Descripción	Se documenta la simulación de caída de al menos un nodo, se analiza el comportamiento del sistema ante cada Consistency Level (ONE, QUORUM, ALL) y se presentan métricas comparativas de disponibilidad y rendimiento con conclusiones.	No se simuló la caída de nodos, el análisis de Consistency Levels es insuficiente, carece de métricas comparativas o las conclusiones no están respaldadas por datos medidos.	2.4

			Rango del punteo	5 - 3	2 - 0	
	Sub-Total de Puntos	60				
	Total	100				